

TEST PLANNING

- A well-defined test plan is essential for successful testing. It acts as a roadmap for the entire testing project.

Preparing a Test Plan:

- A test plan should cover:
- **Scope:** What will and will not be tested.
- **Strategy:** How testing will be performed, including tasks and methods.
- **Resources:** Required hardware, software, and personnel.
- **Timeline:** Schedule for testing activities.
- **Risks:** Potential issues and mitigation plans.

Scope Management: Deciding Features to be Tested/Not Tested:

Scope management involves:

- Understanding the product release.
- Breaking down the release into features.
- Prioritizing features for testing.
- Deciding which features to test and which to exclude.
- Estimating resources needed.

Prioritization factors:

- New and critical features.
- Features with catastrophic failure potential.
- Complex features.
- Extensions of defect-prone features.
- It's crucial to identify features and combinations that *won't* be tested, with justifications and risk assessments.

Deciding Test Approach/Strategy:

- This involves determining the appropriate types of testing for each feature or combination, including:
- Functional testing methods.
- Configurations and scenarios.
- Integration testing.
- Localization validations.
- Non-functional tests.

Setting up Criteria for Testing:

- **Entry Criteria:** Requirements that must be met before testing can begin.
- **Exit Criteria:** Conditions that indicate testing is complete.
- **Suspension Criteria:** Reasons for temporarily halting testing.
- **Resumption Criteria:** Conditions for restarting suspended tests.

- **Identifying Responsibilities, Staffing, and Training Needs:**
- Define roles and responsibilities clearly, ensuring accountability and preventing overlap. Staffing should be based on effort estimation, and training should address skill gaps.

- **Identifying Resource Requirements:**

- Estimate hardware, software, tools, licenses, and environmental needs (office space, support functions). Accurate estimation is crucial to avoid delays and demotivation.

- **Identifying Test Deliverables:**

- Define the outputs of the testing process, including:
 - Test plans (master and individual).
 - Test case specifications.
 - Test cases (including automation).
 - Test logs.
 - Test summary reports.
 - Defect repository updates.

Testing Tasks: Size and Effort Estimation:

- **Size Estimation:** Quantifies the amount of testing. Factors include product size (LOC, function points, screens/reports), automation extent, and platforms/environments. Use a Work Breakdown Structure (WBS).
- **Effort Estimation:** Determines the work required. Factors include productivity data, reuse opportunities, and process robustness. Expressed in person-days/months/years

Activity Breakdown and Scheduling:

- Translates effort into a timeline. Steps include:
- Identifying dependencies (internal and external).
- Sequencing activities.
- Estimating time for each activity.
- Monitoring progress.
- Rebalancing schedules and resources.

Communications Management:

- Establish procedures for effective communication, which will be discussed further in Section 15.3.

Risk Management:

- A cyclical process involving:
- Risk identification (checklists, history, networking).
- Risk quantification (probability and impact).
- Risk mitigation planning (alternative strategies).
- Risk response (when risks occur).

- Common testing risks include:
- Unclear requirements.
- Schedule dependence.
- Insufficient testing time.
- "Show stopper" defects.
- Lack of skilled testers.
- Inability to acquire test automation tools.
- **Summary:**
- Effective test planning is paramount for successful software testing. It requires a thorough understanding of the product, careful consideration of various factors, and proactive risk management. A well-defined test plan ensures that testing is focused, efficient, and contributes to a high-quality product release.

TEST PROCESS

Putting Together and Baselining a Test Plan:

- The test plan consolidates all planning aspects into a single document. A template is typically used and customized for each project.
- The plan undergoes review and approval by designated authorities.
- It's then baselined in the configuration management repository, serving as the foundation for the testing project. Changes to the plan require updates and re-baselining. Templates are also periodically reviewed and updated.

Test Case Specification:

- Based on the test plan, test case specifications are designed. A test case formally consists of:
 - 1.Purpose:** What feature/part is being tested (using consistent naming conventions).
 - 2.Items Under Test:** Versions/releases of the items.
 - 3.Environment:** Hardware, software (OS, database), and product setup (installation, configuration, data).
 - 4.Input Data:** Specific, unambiguous values for each field (e.g., "enter 789" instead of "a three-digit positive integer"). Use files for automated testing.
 - 5.Execution Steps:** Detailed instructions (for manual testing) or scripts (for automated testing). Detail level should match tester skill.
 - 6.Expected Results:** What the user should see (GUI, reports) and changes to persistent storage.
 - 7.Result Comparison:** "Intelligent" comparison of actual vs. expected results, accounting for acceptable differences (e.g., date, ID).
 - 8.Relationships to Other Tests:** Dependencies or reuse possibilities.

- **Update of Traceability Matrix:**

- The requirements traceability matrix is updated with test case identifiers, ensuring two-way mapping between requirements and tests.

- **Identifying Possible Candidates for Automation:**

- Decide which tests to automate based on:

- 1.Repetitive nature of the test.
- 2.Automation effort.
- 3.Manual intervention required.
- 4.Cost of the automation tool.

- **Developing and Baselineing Test Cases:**

- Test cases are developed from specifications. This involves scripting for automated tests and detailed instructions for manual tests.
- Test cases should include change history documentation (what, why, who, when, how, affected files).
- All test artifacts (scripts, inputs, outputs) are stored in the test case database and SCM, requiring review and approval before baselineing.

Executing Test Cases and Keeping Traceability Matrix Current:

- Test cases are executed at appropriate times (e.g., smoke tests daily, system tests during system testing).
- The defect repository is updated with fixed defects and new defects. It's the primary communication vehicle between test and development teams.
- Tests may be suspended based on defined criteria and resumed when conditions are met. Entry and exit criteria apply.
- The traceability matrix is updated during test design and execution. The matrix itself is under configuration management.

Collecting and Analyzing Metrics:

- Test execution data is collected and transformed into meaningful metrics .

Preparing Test Summary Report:

- A report summarizing the test cycle is generated, providing insights into product fitness for release.

Recommending Product Release Criteria:

- The testing team informs management about:
 - 1.Existing defects.
 - 2.Severity and impact of defects.
 - 3.Risks of releasing with these defects.
- Management then makes a business decision about release, balancing the risks of releasing with defects against the risks of delaying the release to fix more defects. "Over testing" to remove every last defect can be as risky as "under testing."

- This section focuses on test reporting, a crucial communication mechanism within the software development process.

TEST REPORTING

- Effective communication is essential during testing. Two main types of reports are used:
 - Test Incident Reports
 - Test Summary Reports (Test Completion Reports)

Test Incident Report:

- These reports are generated throughout the testing cycle whenever defects are found.
- They are essentially entries in the defect repository.
- Each defect has a unique ID, which is used to track the incident.
- High-impact incidents are highlighted in the test summary report.

Test Cycle Report:

- Generated at the end of each test cycle (a series of tests using a specific product build), these reports provide:
 - 1.Summary of activities performed during the cycle.
 - 2.Defects uncovered, categorized by severity and impact.
 - 3.Progress on defect fixes from the previous cycle.
 - 4.Outstanding defects.
 - 5.Variations in effort or schedule compared to the plan.

Test Summary Report:

- This report summarizes the results of a test cycle and provides a release recommendation. Two types exist:
 1. Phase-wise test summary: Generated at the end of each testing phase.
 2. Final test summary (Release Test Report): Contains details from all testing phases and teams.

- A summary report should include:
 1. Summary of activities performed during the cycle/phase.
 2. Variance from planned activities:
 1. Tests planned but not run (with reasons).
 2. Modifications to tests (TCDB should be updated).
 3. Additional tests run.
 4. Differences in planned vs. actual effort and time.
 5. Other deviations from the plan.
 3. Summary of results:
 1. Failed tests with root cause descriptions.
 2. Severity and impact of defects.
 4. Comprehensive assessment and release recommendation:
 1. "Fit for release" assessment.
 2. Release recommendation.

Recommending Product Release:

- The test summary report informs the decision to release the product.
- Ideally, the goal is zero defects, but market pressures may necessitate releasing with some defects.
- Release with defects is acceptable if management believes customer dissatisfaction risk is low.
- This decision is made by senior management after consulting with customer support, development, and testing teams to assess the overall organizational impact. Releasing with low-priority/low-impact defects, or defects triggered by unrealistic conditions, may be acceptable.